

DATA STRUCTURES

UNIT I – BASIC CONCEPTS

This unit introduces the fundamental ideas of **data structures and algorithms**, which form the backbone of efficient problem solving and software development. It focuses on data organization, algorithm design, abstraction, and performance evaluation.

1. Data Objects and Data Structures

Data Objects

A **data object** is a collection of related data elements representing an entity.

Examples:

- Student record (roll number, name, marks)
- Employee details

Data Structures

A **data structure** is a way of organizing and storing data in memory so that it can be accessed and modified efficiently.

Need for Data Structures:

- Efficient data processing
 - Optimal memory utilization
 - Faster access and modification
-

2. Algorithm Specification – Introduction

An **algorithm** is a finite sequence of well-defined instructions to solve a problem.

Characteristics of an Algorithm

- Input
- Output
- Definiteness

- Finiteness
- Effectiveness

Algorithm specification describes the step-by-step logical procedure used to solve a problem.

3. Recursive Algorithms

A **recursive algorithm** solves a problem by calling itself with a smaller input until a base condition is met.

Components of Recursion

- Base case
- Recursive case

Advantages:

- Simple and clear logic

Disadvantages:

- Higher memory usage due to stack calls
-

4. Data Abstraction

Data abstraction refers to providing only essential information while hiding implementation details.

Advantages

- Reduces complexity
- Improves security
- Enhances maintainability

Abstraction is achieved using **Abstract Data Types (ADT)**.

5. Performance Analysis of Algorithms

Performance analysis evaluates the efficiency of an algorithm in terms of time and space.

Time Complexity

Time complexity measures the amount of time an algorithm takes to execute.

Space Complexity

Space complexity measures the amount of memory required by an algorithm.

6. Asymptotic Notations

Asymptotic notations describe the growth rate of algorithms.

Big O Notation (O)

Represents the upper bound (worst case).

Omega Notation (Ω)

Represents the lower bound (best case).

Theta Notation (Θ)

Represents the exact bound.

7. Complexity Analysis – Examples

- Linear search $\rightarrow O(n)$
- Binary search $\rightarrow O(\log n)$
- Bubble sort $\rightarrow O(n^2)$

These examples help compare algorithm efficiency.

8. Linear Data Structures

Linear data structures store elements sequentially.

Examples:

- Arrays
- Linked Lists

- Stacks
 - Queues
-

9. Non-Linear Data Structures

Non-linear data structures organize data hierarchically or graph-based.

Examples:

- Trees
- Graphs

UNIT II – ARRAYS, LINKED LISTS, STACKS AND QUEUES

This unit focuses on **linear data structures** and their representations using arrays and linked lists. It also covers abstract data types (ADT) such as stacks and queues along with their applications.

1. Arrays

An **array** is a collection of elements of the same data type stored in contiguous memory locations.

1.1 Single Dimensional Array

A single dimensional array stores elements in a linear sequence.

Applications:

- Searching and sorting
 - Storing lists of data
-

1.2 Two Dimensional Array

A two dimensional array stores data in row and column format.

Applications:

- Matrix operations

- Tables and grids
-

2. Sparse Matrices

A **sparse matrix** is a matrix in which most of the elements are zero.

2.1 Array Representation of Sparse Matrix

Stores only non-zero elements along with their row and column indices.

2.2 Linked List Representation of Sparse Matrix

Uses nodes to store non-zero elements efficiently, saving memory.

3. Linear List ADT

A **Linear List ADT** is an abstract data type that represents an ordered collection of elements.

3.1 Array Representation of Linear List

Elements are stored sequentially in an array.

Advantages:

- Fast access

Disadvantages:

- Fixed size
-

3.2 Linked List Representation of Linear List

Elements are stored in nodes connected by pointers.

Advantages:

- Dynamic size

Disadvantages:

- Extra memory for pointers
-

4. Singly Linked List

A **singly linked list** consists of nodes where each node contains data and a pointer to the next node.

Operations

- Insertion
 - Deletion
 - Traversal
-

5. Circular Linked List

In a circular linked list, the last node points back to the first node.

Operations

- Insertion
- Deletion

Advantages:

- Efficient traversal
-

6. Doubly Linked List

Each node contains data, a pointer to the previous node, and a pointer to the next node.

Operations

- Insertion
- Deletion

Advantages:

- Bidirectional traversal
-

7. Stack ADT

A **stack** is a linear data structure that follows the **Last In First Out (LIFO)** principle.

Stack Operations

- Push
- Pop
- Peek

Implementation

- Array implementation
 - Linked list implementation
-

Applications of Stack

- Infix to postfix conversion
 - Evaluation of postfix expressions
 - Recursion implementation
-

8. Queue ADT

A **queue** is a linear data structure that follows the **First In First Out (FIFO)** principle.

Queue Operations

- Enqueue
- Dequeue

Implementation

- Array implementation
 - Linked list implementation
-

9. Circular Queue

A circular queue connects the last position back to the first position.

Operations

- Insertion

- Deletion

Advantages:

- Efficient memory utilization
-

10. Polynomial Representation

Polynomials can be represented using arrays or linked lists.

Linked List Representation

Each node stores:

- Coefficient
- Exponent

Advantages:

- Efficient polynomial operations

UNIT III – TREES AND DISJOINT SETS

Trees – Definition

A tree is a non-linear hierarchical data structure consisting of a finite set of nodes, where one node is designated as the root and the remaining nodes form disjoint subtrees. There is exactly one path between the root and every other node.

Tree Terminology

- **Node:** Basic element of a tree containing data.
- **Root:** Topmost node of the tree.
- **Parent:** A node that has one or more children.
- **Child:** A node that descends from a parent.
- **Leaf (External node):** A node with no children.
- **Internal node:** A node with at least one child.
- **Degree of a node:** Number of children of the node.

- **Degree of a tree:** Maximum degree of any node in the tree.
 - **Level:** Number of edges from the root to the node.
 - **Height of a tree:** Maximum level of any node.
 - **Depth of a node:** Length of the path from root to that node.
-

Binary Trees

Definition of Binary Tree

A binary tree is a tree data structure in which each node has at most two children, called the left child and the right child.

Properties of Binary Trees

- Maximum number of nodes at level i is 2^i .
- Maximum number of nodes in a binary tree of height h is $2^{h+1} - 1$.
- Minimum number of nodes in a binary tree of height h is $h + 1$.
- In a full binary tree, number of leaf nodes = number of internal nodes + 1.

Binary Tree ADT

Binary Tree Abstract Data Type defines the following operations:

- Create a binary tree
 - Insert a node
 - Delete a node
 - Search an element
 - Traverse the tree
 - Find height, size, and depth
-

Representation of Binary Trees

Array Representation

- Root is stored at index 1.

- Left child of node at index i is stored at index $2i$.
- Right child of node at index i is stored at index $2i + 1$.

Advantages: Efficient for complete binary trees.

Disadvantages: Wastage of memory for sparse trees.

Linked Representation

Each node contains:

- Data field
- Pointer to left child
- Pointer to right child

Advantages: Efficient memory usage, dynamic structure, easy insertion and deletion.

Binary Tree Traversals

Traversal is the process of visiting all nodes of a tree exactly once.

Preorder Traversal (Root–Left–Right)

- Root node is visited first.
- Used in prefix expression generation.

Inorder Traversal (Left–Root–Right)

- Root is visited between left and right subtrees.
- Produces sorted order in Binary Search Trees.

Postorder Traversal (Left–Right–Root)

- Root is visited last.
- Used in postfix expression evaluation.

Level Order Traversal

- Nodes are visited level by level.
 - Implemented using a queue.
-

Threaded Binary Trees

Definition

A threaded binary tree is a binary tree in which NULL pointers are replaced with special pointers called threads that point to inorder predecessor or successor.

Types

- Left-threaded
- Right-threaded
- Fully threaded (both left and right)

Advantages

- Inorder traversal without recursion or stack
 - Faster traversal
-

Priority Queues

Definition

A priority queue is an abstract data type where each element is associated with a priority, and elements are removed based on priority rather than insertion order.

Applications

- CPU scheduling
 - Job scheduling
 - Dijkstra's shortest path algorithm
 - Event simulation systems
-

Max Priority Queue – Max Heap

Definition of Max Heap

A max heap is a complete binary tree where the value of each node is greater than or equal to the values of its children.

Insertion into a Max Heap

- Insert the element at the last position.
- Compare it with its parent.
- Swap repeatedly (heapify up) until heap property is restored.

Deletion from a Max Heap

- Remove the root element (maximum).
 - Replace root with the last element.
 - Perform heapify down to restore heap property.
-

Disjoint Set ADT

Definition

A disjoint set is a data structure that maintains a collection of non-overlapping (disjoint) sets.

Equivalence Relations

- **Reflexive:** $a \sim a$
 - **Symmetric:** if $a \sim b$, then $b \sim a$
 - **Transitive:** if $a \sim b$ and $b \sim c$, then $a \sim c$
-

Dynamic Equivalence Problem

It involves dynamically determining whether two elements belong to the same set using:

- **Find** operation
 - **Union** operation
-

Basic Data Structure

Each set is represented as a tree.

- Root represents the set.

- Each element points to its parent.
-

Smart Union Algorithms

Union by Rank

- Attach the smaller tree under the root of the larger tree.
- Reduces tree height and improves efficiency.

Path Compression

- During find operation, nodes are made to point directly to the root.
- Flattens the structure of the tree.

Worst Case Analysis

- Union by rank alone: $O(\log n)$
 - Union by rank with path compression: $O(\alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function (almost constant).
-

Application – Generation of Mazes

- Each cell in the maze is treated as a disjoint set.
- Walls are removed by performing union operations between adjacent cells.
- Ensures there is exactly one path between any two cells.
- Disjoint sets help prevent cycles during maze generation.

UNIT IV – SEARCHING, SORTING, AND EXTERNAL SORTING

SEARCHING

Linear Search

Linear search is the simplest searching technique in which each element of the list is checked sequentially until the desired element is found or the list ends.

Procedure:

- Start from the first element.
- Compare each element with the key.
- Stop when the key is found or all elements are checked.

Characteristics:

- Works on both sorted and unsorted data.
- Easy to implement.
- Inefficient for large datasets.

Time Complexity:

- Best case: $O(1)$
 - Average case: $O(n)$
 - Worst case: $O(n)$
-

Binary Search

Binary search is an efficient searching technique that works only on sorted lists. It repeatedly divides the search interval into two halves.

Procedure:

- Compare the key with the middle element.
- If equal, search ends.
- If key is smaller, search left half; if larger, search right half.
- Repeat until found or range becomes empty.

Characteristics:

- Requires sorted data.
- Faster than linear search.
- Can be implemented iteratively or recursively.

Time Complexity:

- Best case: $O(1)$
- Average case: $O(\log n)$

- Worst case: $O(\log n)$
-

HASHING

Introduction to Hashing

Hashing is a technique used to map keys to specific locations in a table using a hash function for fast data access.

Hash Table

A hash table is a data structure that stores key–value pairs using a hash function to compute an index into an array.

Advantages:

- Fast access and insertion.
 - Efficient for large datasets.
-

Hash Functions

A hash function converts a key into an index in the hash table.

Properties of a Good Hash Function:

- Uniform distribution of keys.
- Minimal collisions.
- Simple and fast computation.

Common Hash Functions:

- Division method: $h(k) = k \bmod m$
 - Mid-square method
 - Folding method
-

Overflow Handling (Collision Resolution)

Collision occurs when two keys hash to the same index.

Techniques:

- **Linear probing:** Search next available slot.
 - **Quadratic probing:** Use quadratic increments.
 - **Double hashing:** Use a second hash function.
 - **Chaining:** Store multiple elements at the same index using linked lists.
-

Comparison of Searching Methods

Method	Data Order Required	Time Complexity	Suitable For
Linear Search	No	$O(n)$	Small data
Binary Search	Yes	$O(\log n)$	Large sorted
Hashing	No	$O(1)$ average	Fast access

SORTING

Insertion Sort

Insertion sort builds the sorted list one element at a time by inserting each element into its correct position.

Characteristics:

- Simple and stable.
- Efficient for small datasets.
- Adaptive in nature.

Time Complexity:

- Best case: $O(n)$
 - Average/Worst case: $O(n^2)$
-

Selection Sort

Selection sort repeatedly selects the smallest element from the unsorted portion and places it in the correct position.

Characteristics:

- Simple but inefficient.
- Not stable.
- Number of swaps is minimal.

Time Complexity:

- Best/Average/Worst: $O(n^2)$
-

Radix Sort

Radix sort sorts numbers digit by digit starting from least significant digit to most significant digit using a stable sub-sort.

Characteristics:

- Non-comparison-based sorting.
- Efficient for integers and strings.

Time Complexity:

- $O(d(n + k))$, where d = number of digits
-

Quick Sort

Quick sort is a divide-and-conquer algorithm that selects a pivot element and partitions the array around the pivot.

Characteristics:

- Very fast in practice.
- In-place sorting.
- Not stable.

Time Complexity:

- Best/Average: $O(n \log n)$
 - Worst case: $O(n^2)$
-

Heap Sort

Heap sort uses a binary heap data structure to sort elements.

Procedure:

- Build a max heap.
- Repeatedly delete the maximum element.

Characteristics:

- In-place sorting.
- Not stable.

Time Complexity:

- Best/Average/Worst: $O(n \log n)$
-

Merge Sort

Merge sort divides the array into halves, sorts them recursively, and merges them.

Characteristics:

- Stable sorting.
- Requires extra memory.
- Suitable for large datasets.

Time Complexity:

- Best/Average/Worst: $O(n \log n)$
-

Comparison of Sorting Methods

Algorithm Time Complexity Stable In-place

Insertion	$O(n^2)$	Yes	Yes
Selection	$O(n^2)$	No	Yes
Radix	$O(nk)$	Yes	No

Algorithm Time Complexity Stable In-place

Quick	$O(n \log n)$ avg	No	Yes
Heap	$O(n \log n)$	No	Yes
Merge	$O(n \log n)$	Yes	No

EXTERNAL SORTING

Model for External Sorting

External sorting is used when data does not fit into main memory and must be stored on secondary storage.

Components:

- Main memory (limited)
 - Secondary storage (disk)
 - Input and output buffers
-

Basic External Sorting Algorithm

- Divide data into manageable blocks.
 - Sort each block in memory.
 - Write sorted blocks (runs) to disk.
 - Merge runs until a single sorted file is obtained.
-

Multi-way Merge

- Merges more than two sorted runs simultaneously.
 - Uses multiple buffers.
 - Reduces number of merge passes.
-

Polyphase Merge

- Uses unequal number of runs on tapes/disks.
 - Reduces the number of dummy runs.
 - Improves efficiency over balanced merge.
-

Replacement Selection

- Produces runs longer than memory size.
- Uses a priority queue (min heap).
- Improves performance of external sorting.

UNIT V – GRAPHS AND SEARCH TREES

GRAPHS

Definition of Graph

A graph is a non-linear data structure consisting of a finite set of vertices (nodes) and a set of edges connecting pairs of vertices. A graph is represented as $G = (V, E)$ where V is the set of vertices and E is the set of edges.

Graph Terminology

- **Vertex (Node)**: Basic unit of a graph.
- **Edge**: Connection between two vertices.
- **Directed graph (Digraph)**: Edges have direction.
- **Undirected graph**: Edges have no direction.
- **Weighted graph**: Edges have weights or costs.
- **Unweighted graph**: Edges have no weights.
- **Degree of a vertex**: Number of edges incident to the vertex.
- **Indegree**: Number of incoming edges to a vertex.
- **Outdegree**: Number of outgoing edges from a vertex.
- **Path**: Sequence of vertices connected by edges.

- **Simple path:** Path with no repeated vertices.
- **Cycle:** Path that starts and ends at the same vertex.
- **Connected graph:** Path exists between every pair of vertices.
- **Complete graph:** Every pair of vertices is connected by an edge.
- **Subgraph:** Graph formed from a subset of vertices and edges.

Applications of Graphs

- Computer networks and routing
- Social networks
- Transportation systems
- Web page linking
- Project scheduling
- Electrical circuits

Properties of Graphs

- In an undirected graph, sum of degrees of all vertices = $2 \times$ number of edges.
- A connected graph with n vertices has at least $n - 1$ edges.
- A complete graph with n vertices has $n(n - 1)/2$ edges.
- A tree is a connected acyclic graph.

Graph ADT

The Graph Abstract Data Type defines:

- Create a graph
- Add and remove vertices
- Add and remove edges
- Check adjacency of vertices
- Traverse the graph
- Find degree of a vertex

Graph Representations

Adjacency Matrix

- A 2D array of size $V \times V$.
- Entry is 1 (or weight) if edge exists, else 0.

Advantages:

- Simple representation
- Fast edge lookup

Disadvantages:

- High memory usage for sparse graphs
-

Adjacency List

- Array of linked lists
- Each vertex stores list of adjacent vertices

Advantages:

- Efficient memory usage
- Suitable for sparse graphs

Disadvantages:

- Edge lookup is slower than matrix
-

Graph Search Methods

Depth First Search (DFS)

DFS explores as far as possible along each branch before backtracking.

Method:

- Uses recursion or stack
- Start from a vertex

- Visit an unvisited adjacent vertex recursively

Applications:

- Detect cycles
 - Path finding
 - Topological sorting
-

Breadth First Search (BFS)

BFS explores vertices level by level.

Method:

- Uses a queue
- Visit all adjacent vertices before moving to next level

Applications:

- Shortest path in unweighted graphs
 - Level-order traversal
-

Complexity Analysis

- DFS time complexity: $O(V + E)$
 - BFS time complexity: $O(V + E)$
 - Space complexity depends on representation
-

SEARCH TREES

Binary Search Tree (BST)

Definition

A Binary Search Tree is a binary tree in which:

- Left subtree contains values less than the root
- Right subtree contains values greater than the root

- Both subtrees are also BSTs
-

Binary Search Tree ADT

Operations include:

- Search
 - Insert
 - Delete
 - Traverse
 - Find minimum and maximum
-

BST Operations

Searching

- Compare key with root
- Traverse left or right accordingly
- Continue until found or NULL

Insertion

- Insert new node at leaf position maintaining BST property

Deletion

Three cases:

- Deleting a leaf node
 - Deleting a node with one child
 - Deleting a node with two children (replace with inorder successor)
-

BALANCED SEARCH TREES

AVL Trees

Definition

An AVL tree is a self-balancing binary search tree where the height difference (balance factor) between left and right subtrees is at most 1.

Examples

- LL rotation
 - RR rotation
 - LR rotation
 - RL rotation
-

B-Trees

Definition

A B-tree is a self-balancing multi-way search tree where each node can have multiple keys and children.

Examples

- Used in databases and file systems
 - Maintains sorted data with logarithmic height
-

Red-Black Trees

Definition

A red-black tree is a self-balancing binary search tree with nodes colored red or black and specific balancing rules.

Examples

- Root and leaves are black
 - Red nodes cannot have red children
-

k-d Trees

Definition

A k-dimensional tree is a space-partitioning data structure used for organizing points in k-dimensional space.

Applications:

- Range searching
- Nearest neighbor search

Comparison of Search Trees

Tree Type	Balanced	Height	Use Case
BST	No	$O(n)$	Simple searching
AVL	Yes	$O(\log n)$	Fast lookup
Red-Black	Yes	$O(\log n)$	Balanced performance
B-Tree	Yes	$O(\log n)$	Disk-based storage
k-d Tree	Yes	$O(\log n)$	Multidimensional data